

Comparative study of neural architecture search methods: random search, RNN + Reinforcement learning, and evolutionary algorithms on CIFAR-10

Jiadong Liu

Taishan University, Tai'an, China, 271000

loft233@163.com

Abstract. In the past few years, Neural Architecture Search (NAS) is an attractive technique that promised automatic design for high-performance neural networks. However, NAS methods considerably vary in search efficiency, as well as model performance. In this paper, we have compared three mainstream NAS methodologies: Random Search, RNN + RL (Recurrent Neural Networks and Reinforcement Learning based method), and the Evolutionary Algorithm suggested in this paper. The experiments were on the CIFAR-10 dataset and each method's accuracy, training time, and parameter count were analyzed. Research methodology further involves experimental analysis of these three NAS methods over the CIFAR-10 dataset, with particular attention proposed to enhance the performance of the Evolutionary Algorithm through improving the encoding and mutation strategies. The Evolutionary Algorithm has good searching efficiency and is hence deemed 'environmentally friendly'. RNN + RL is significantly better model- and parameter-wise. Implementing a dynamic mutation rate strategy with the Evolutionary Algorithm enhances the searching efficiency and model performance of the Evolutionary Algorithm. Successive to mark the evolution of fine-tuning of the Evolutionary Algorithm with other NAS techniques will further enhance it, finally unleashing its full potential in advancing automated design techniques for deep learning models.

Keywords: Neural architecture search, evolutionary algorithm, reinforcement learning, deep learning, model optimization

1. Introduction

Researches are focused on how to come up with a highly efficient and high-performance neural network architecture since it is a very important aspect of deep learning that has been applied in various fields. Almost all the conventional methods of designing are manual, based on expert experience with much experimentation. It ends up becoming laborious and ineffective, especially when working with complex and huge landscapes of searches. The current innovation is known for them. Evolution of an architecture search framework for designing network architectures that uses algorithmic exploration and tuning to enhance the efficiency and efficacy of model development is a significant innovation in the current scenario[1].

Existing architectural design process redundancy in human expertise will be reduced by NAS, bringing such novel and under manual operation highly efficient architectures into focus which

otherwise may go unnoticed[2]. However, the different NAS techniques present notable diversity concerning search efficiency, computational cost, and model quality. Understanding these diversities is important to choose the right NAS technique that is appropriate for certain application requirements and computational constraints[3].

Comparison and performance analysis of three of the popular NAS methods, Random Search, RNN + RL (Neural network RNN together with Reinforcement Learning), and Evolutionary Algorithm (An enhanced evolutionary search algorithm for improved search efficiency and model performance via a dynamic mutation rate strategy) proposed in this work using CIFAR-10 as a benchmark are presented. These selected methodologies depict architectural issues in different ways:

- (1)Random search: Network architectures are produced by randomly sampling in a predefined search space and examining the performance of each architecture.
- (2)RNN + Reinforcement Learning: An RNN-based controller is used to sequentially generate architecture components. The controller is optimized through reinforcement learning for enhanced performance.
- (3) Evolutionary Algorithm: Network architectures are evolved by simulating processes of biological evolution, i.e., selection, crossover, and mutation. As one novelty, in this study a dynamic mutation rate strategy has been proposed towards better search efficiency and model performance.
- (4) This work will apply the three NAS methods to practical applications to architecture search strategies systematically and compare them on the CIFAR-10. The emphasis is to compare these methods by their performance on metrics like model accuracy, training time, and parameter efficiency, thereby indicating pros and cons of each methodology.

2. Literature Review

So far, the research in NAS started with looking for the complicated architectural spaces using Reinforcement Learning (RL) and Evolutionary Algorithms (EA) as the main explorers. The first of these was the technique that began iteratively proposing candidate architectures that could be generated using RNN-based controllers and then transformed such refinements by RL-driven reward signals into a great leap over manual design [4]. Therefore, the RNN framework offered a more systematic way of unveiling high-performance models, but often encountered large computational overhead and low search efficiency, which called for alternatives to be investigated. The computation bottleneck was then tried to be resolved through various algorithmic optimizations and heuristic enhancements, which further loosened the situation later on with simplicity and efficiency in the search process [5]. In that sense, the evolutionary algorithm may be viewed as a population-based metaheuristic technique or as a strategy, alternative or supplemental to deterministic learning algorithms that allow balancing adequately the exploration/exploitation trade-off in a principled way. The empirical success of evolutionary algorithm-based techniques can be mainly attributed to their capability of automatically refining initial candidate solutions and exploiting both convex and non-convex structure information related to system architecture in deriving good approximate analytical models out of input-output data[6].

Beyond RL and EA, NAS literature includes other strategies like greedy searching, optimization based on gradients [7], and performance prediction [8] along with the use of optimization techniques[9] that reduce search space or introduce search space pruning to enhance the speed and accuracy of the search. Currently, most NAS research focuses on constricted yet strategically selected search spaces to lighten the computational load while ensuring quality. Better strategies and improvements in applications helped NAS get architectures that surpassed not only existing standard benchmarks but also resource-constrained environments. Quite a breakthrough in the NAS field was the transition from pure RL solutions to diversified and more effective hybrids involving evolutionary algorithms as well as optimization of search space techniques. This brought NAS closer to real applicative scenarios and automatic machine learning pipelines.

3. Overview of neural architecture search methods

3.1. *Random search*

The next method of learning is Random Search[10]; it generally randomly samples network architectures within a predefined search space. In essence, this technique doesn't depend on any strategy but generates neural network architectures by randomly applying all possible combinations to test the performance of a particular configuration.

3.2. *RNN + Reinforcement learning method*

The controller based on recurrent neural network architecture is supposed to produce network architecture sequences. According to the authors, this architecture's controller is optimized through appropriate reinforcement learning methods, thus progressively coming up with architecture that is better suited to achieving the targeted performance[11]. RNN generates the architecture configurations with the reinforcement learning reward signal updating the RNN controller's parameters according to the architecture performance. Only the input layer's dimensions are assumed known, so that the parameter space in problem 1 with 3 hidden layers and the output layer containing 1 element can be defined as 4-tuple $\langle \theta, \theta, \theta, \theta \rangle$.

3.3. *Evolutionary algorithm method*

An Evolutionary Algorithm for optimising neural network architectures[12] which works by approximating the mechanisms of biological evolution like selection, crossover, and mutation. The well-performing individuals are retained in each generation, and new ones are created through crossover, with mutation operations added towards enhancing population diversity. In this study, we propose to dynamically control the mutation rate, using a high rate of mutation in early stages for driving diversity and a low rate of mutation later to assist convergence.

4. Methodology

4.1. Experimental setup

4.1.1. **Dataset. (1). Dataset Selection**

We use the CIFAR-10 dataset, a standard dataset commonly used for image classification tasks.

(2)Dataset Scale

Training Set: 50,000 color images of 32×32 pixels

Test Set: 10,000 color images of 32×32 pixels

4.1.2. *Experimental Environment.* (1)Hardware Environment

Device: NVIDIA GeForce RTX 4090

(2)Software Environment

Operating System: Windows

Programming Language: Python 3.11.5

Deep Learning Framework: PyTorch 2.0.1+cu118

Other Dependencies: torchvision 0.15.2+cu118, numpy, matplotlib, tqdm, psutil

4.2. *Network architecture encoding and implementation*

4.2.1. *Specific implementation of the randomized search method*

By randomly generating network architectures, we can efficiently explore various possible configurations and evaluate their performance on specific tasks.

Experimental Setup

To guarantee the reproducibility of our results, we set up the following: Random seed: set the random seed value to the same (RANDOM_SEED = 42) for Python random module, NumPy, and PyTorch. Device selection: tested for GPU availability and, if available, the model training was done on the GPU to speed up computation.

In the random search, 500 iterations were carried out. Each consisted of a 5-layer network architecture in which each layer may be of 5 possible choices of operations. Model training parameters: Training epochs: 20, batch size: 256, Optimizer: SGD, learning rate: 0.01, momentum: 0.9, weight decay: $5e-4$, loss function: CrossEntropyLoss. The CIFAR-10 dataset has 50,000 training images and 10,000 test images with the size of 32×32 -pixel images. and for the training set data augmentation, was done by random crops left-right flips and rotations. set to 0; unit variance set for having unit variance; set to UNIT variance for having unit variance.

(1) In every iteration, we will generate a random permutation of integers corresponding to different kinds of network layers. We later will describe this as generating an architecture. This randomness is helping us look at various regions in search space. We dynamically build the corresponding neural network model based on the generated architecture sequence. In constructing the model, each layer is added as per the indication in the sequence such that they are properly connected with each other. We will be monitoring the feature map size at model construction. The layer for the feature map size is too small, or for some invalid values, is attaining an incomplete feature map size, then the model is invalid. The architecture has to be generated all over again.

(2) The model is trained using the training set. the loss function being the cross-entropy loss function and the optimizer being SGD. to keep a tap on overfitting, the learning rate is adapted dynamically with the help of cosine annealing learning rate scheduler. after the training is completed, the performance of the model is tested on the test set and the main measure of it is accuracy. the accuracy with which the model is evaluated is going to be used for gauging the best model to direct the further optimization process.

(3) The above model generation, validation, training, and evaluation process is repeated for 500 iterations. In each iteration a new stochastic architecture is tried to make sure we have not found the best possible model simply because of bad luck in the choice of architecture. After every such comparison, if the model accuracy at present is better than the best recorded one, then we update the record for the best model specifying its architecture, number of parameters, and the accuracy.

4.2.2. The RNN-based architecture generation approach

The RNN-based architecture generation approach utilizes a network of controllers to generate a sequence of architectures for the model and optimizes the parameters of the controllers through reinforcement learning.

Experimental Setup

In this approach, we used the generator just as a controller network to be trained towards learning good sequences of architectures. The exact random seed value, 42, was used for reproducibility. It's a one-layer RNN with a hidden layer size of 32, which gave 5 layers of architectural sequences from 5 possible operations, with an embedding dimension of 16. Cross-entropy between generated sequence of architectures and final optimal model accuracy associated with that architecture, using Adam optimizer with a learning rate of 0.001 for 5 epochs, was used to train the controller. This is because it takes the moving average accuracy baseline with a momentum of 0.9 to stabilize the update of policy gradient. The model was trained with 20 epochs and a batch size of 256 examples for SGD with a learning rate of 0.01, and having CrossEntropyLoss. Data preprocessing for the training set includes random crops, flips, and rotations; normalization is done on both training and test set. The test set accuracy of it is what tells how well the above model has been trained. In other words, it is the reward signal through policy gradient to update model parameters of the controller based on architect accuracy. The function of the controller sample_architecture is to generate the architecture, which should now have its evaluation done by the same functions build_model and train_and_evaluate like

the stochastic search approach; this is how the RNN controller is enabled to optimize for generating architectures.

(1) The architecture sequence is generated by the RNN controller [13]. That is, at each time step, the architecture of the next layer is predicted by an LSTM layer and a fully connected layer within the type of network layer that is being referred to is represented as an integer within the sequence. Softmax is applied to the output during the prediction processes to convert it into probabilities. The corresponding architecture layers are sampled based on those probabilities from a multinomial distribution. These are trained by optimizing their parameters with the policy gradient approach in reinforcement learning so that the quality in the generated architectures is enhanced in line with the performance (accuracy) of architectures on a test set.

(2) The controller generates a sequence of architectures and then passes this sequence to the function `build_model` along with the input data to actually build the model. The model is then trained on the training data and tested for its accuracy on a test set. Such accuracy works as the reward signal for setting up the loss that optimizes the controller parameters by the loss function. Namely, we are trying to train the controller well enough so that it gives better performing architectures in future. The training of the controller involves the generation of architectures, training of models, computation of the reward signal (accuracy), and updating of the controller's parameters using the policy gradient method. We are not intending to show the entire training code here, but this flow constitutes the controller's optimization process.

(3) The RNN controller is initialized with specific input, hidden, and output sizes (e.g., a hidden layer size of 32 containing 1 layer of RNN). This controller is trained using the Adam optimizer with a learning rate of 0.001. The controller is, over multiple training cycles, continuously generating new architectures. If after each evaluation the new architecture, beats the best current one, then the best is updated and saved.

(4) Just like in the first part is `_valid_model` for checking if the model can handle input data is called. Further calculate the number of parameters in the model, and save the best model.

4.2.3. *Architecture Generation Based on Evolutionary Algorithms*

Evolutionary algorithms create and optimize through population evolution neural network architectures for the search for optimal network configurations. We present in this section a dynamic mutation rate strategy: With the aim to make evolutionary algorithms more efficient in building the architecture of a neural network, we put forward a dynamic mutation rate strategy. Conventionally, evolutionary algorithms use a fixed mutation rate which hampers a proper balancing act between the exploration of new solutions and the exploitation of known good solutions; therefore, it results in a slow search. This dynamic mutation rate strategy aims to address this problem: At the beginning of the algorithm (the exploration stage): apply a high mutation rate in order to increase the diversity of the population, realize an extensive exploration of the search space, and avoid getting stuck in local optimality. Towards the end of the algorithm (the development stage): with the increase in evolutionary generations, decrease the mutation rate gradually to tamp down randomness and enhance an in-depth optimization of fine individuals, and speed up the convergence.

The evolutionary algorithms set free and optimize architectures of the neural network, searching for the same optimum population evolution process.

We propose: a dynamic mutation rate strategy for bettering the performance of evolutionary algorithms in searching neural network architectures. Traditional evolutionary algorithms adopt a fixed mutation rate, which can not balance the relations of exploring new solutions with the use of known good solutions, therefore realizing inefficient search. In this work, we attempt to solve this problem by applying the dynamic mutation rate strategy: early stage of the algorithm (discovery stage): set the mutation rate relatively higher to encourage population diversity, general exploration of the search space, and steer clear of local optima. late stage of the algorithm (convergence stage): as the evolutionary generations increase, progressively reduce the mutation rate to increase determinism, intensify the deep optimization of great individuals, and hasten convergence.

(1)To keep the experiments reproducible, we set a random seed and checked GPU availability for speeding up the model training. Models are built from architecture sequences, where each integer corresponds to some type of network layer (for instance, it could be a convolutional layer or pooling layer). If the feature map size is too small at build time, then that model will be deemed as not valid.

(2)Model training is performed by cross-entropy loss with SGD optimizer, and the learning rate is dynamically changed by the cosine annealing learning rate scheduler. What happens at this stage is that the model is basically trained for doing forward propagation—calculating losses and then updating parameters. After training the model, the same is evaluated on a test set, and checking accuracy is done to see the performance.

(3)The efficiency of evolutionary algorithms typically drives upon the accuracy with which the individuals can define the fitness of the model. At first, the population is sorted based on fitness and the best performing individuals are retained (this is known as the elite strategy) [14]. Then, a dynamic mutation rate strategy mutes the elite individuals to generate new individuals until the population size is achieved. Elite manning is sure while, on the other hand, it allows diversity through dynamic mutation hence likelihood of discovering superior architectures is increased.

(4)Main program flow:First of all, the population is initialized. Then the initial population is established by generating multiple architectural sequences randomly. After this, process of iterative evolution begins. To begin with, at each generation, evaluate the fitness of all individuals in the population. Just construct, train, and evaluate the model for accuracy on this individual of the population. Next, a new population will be created as follows. evolve_population takes the best performing elite individuals and uses a dynamic mutation rate strategy to create new individuals that populate it up to some predefined size. It should also dynamically mutate with time concerning the generation number. The best architecture is retrained from scratch using all examples on mutation with probability changing weights. That is to say, the best architecture found is retained along with its accuracy and model in each generation. It retains only the best model.

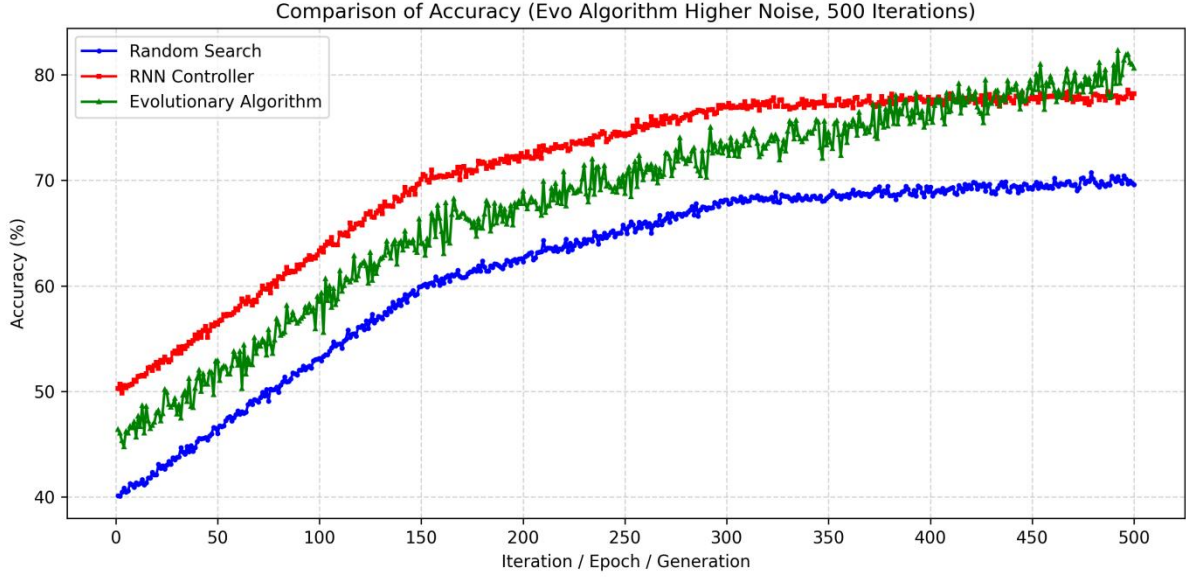
(5)Lastly, at the end of each iteration, verify if the models produced point to the is_valid_model function so the models' architecture can be accurately trained and tested. Meanwhile, calculate the number of parameters of each model so that complexity is understood by the model. This information helps in handling the trade-off between performance and resource consumption, which is a guideline for the subsequent architecture optimization process.

4.3. Experimental results presentation

Table 1. Presents the performance of the three methods in Neural Architecture Search (NAS), showing their differences in terms of accuracy, parameter count, and training time.

Table1 1Method	Best Model Accuracy	Best Model Architecture	Number of Parameters	Total Training Time
Random Search	70.24%	Activation Function: ReLU ('conv', 16, 3), ('pool,), ('conv', 16, 3)	47.982	19minutes28.57seconds
RNN Controllor	78.10%	[3, 1, 3, 2, 3]	341.207	21minutes31.74seconds
Evolutionary Algorithm	80.02%	[0, 0, 3, 0, 2]	162.385	7minutes 59.52seconds

Figure 1. Variation of model accuracy at different stages.



iterations in Random Search

Accuracy vs iteration after 500 iterations for random search, RNN controller, and evolutionary NAS methods is depicted in Fig. 1. As evident from the figure, the evolutionary algorithm exhibits the highest variation in the final iterations but attains the highest accuracy value. While the RNN controller's curve is relatively smoother than that of the evolutionary algorithm, it converges to a moderate high level of accuracy; random search's curve is noticeably low, portraying its inefficiency in the search strategy.

5. Discussion

Performance and efficiency vary significantly from one NAS method to another as shown by the experimental results for random search, RNN controller, and evolutionary algorithms. The RNN controller and evolutionary algorithm both seem good at finding better network architectures than random search. However, the evolutionary algorithm is the most efficient and effective way to do this.

Concerning the general performance and the usage of resources, it may be said that the best final accuracy of 80.02% was reached by the evolutionary algorithm requiring the shortest training time of 7 minutes 59.52 seconds. Even with discrepancies later in the stages, the evolutionary algorithm fastly converges toward superior architectures, affirming that it is an efficient search process and can quickly and optimally cross an architectural space. Such robustness would also suggest that an evolutionary strategy of adaptive mutation and selection may correctly trade off between exploration and exploitation and more quickly identify good architectures.

The controller based on RNN gives moderate high accuracy, 78.10% to be exact, but it demands longer training time; 21 minutes and 31.74 seconds, compared with other approaches, and depends on using reinforcement learning to gradually improve the controller's strategy. Better than random search, the longer training time and larger number of parameters are the main reasons to suspect that its method search would be less computationally intensive.

Holistic random search, however, provided the worst trade-offs, with the lowest accuracy (70.24%) and relatively long training time (19 minutes 28.57 seconds). The totally random choice of models without any particular heuristics led to very slowly improving models, ultimately putting down most of the results in context. Hence, it is really necessary to have a systematic or adaptive search strategy to enhance efficiency as well as model performance.

6. Conclusion

In this study, we have examined the performance of three NAS techniques: random search, RNN controller, and an evolutionary algorithm with a dynamic mutation rate strategy. The results have shown that the evolutionary algorithm records the highest accuracy, which is 80.02% with the RNN controller coming second at 78.10%. Both scores are much better than random search's poor accuracy of 70.24% as calculated over the test set. To be explicit, our evolutionary algorithm and the dynamic mutation rate strategy outperform the RNN controller in the aspect of accuracy and shine with an excellent edge in training time; they took only seven minutes and 59.52 seconds to train. Although it experienced some difficulty in handling certain architectures, the evolutionary algorithm still pulled off an excellent accuracy rating as well as fast convergence rate. Comparatively, random search took much longer, i.e. 19 minutes 28.57 seconds, and was less accurate.

These results point out the structural efficiency of evolutionary algorithms and model performance; especially, it is seen that a dynamic mutation rate strategy provides an important advantage with respect to both. Furthermore, the guidance mechanism tends to find better structures at higher computational costs than random search, stressing that the guided search mechanism significantly enhances model accuracy and reduces computation in refining search strategies. Thus, the efficacy of evolutionary algorithms and their dynamic mutation rate strategy stands adequately demonstrated and the further enhanced future application of evolutionary algorithms in NAS is suited with resource-sensitive environments. In automated machine learning pipelines, currently an important prerequisite to developing efficient ways for developing deep learning models [15], NAS hears the clarion call for better ways.

An issue of this study is that it is limited by the scale at which the process can be applied since, due to hardware and time constraints, only a limited search can be allowed by these methods. Particularly with evolutionary algorithms, as all candidate architectures were not treated in the same way, may slightly lead to not right as much written above results [16]. Further studies on algorithmic improvements could include scaling the experiments to higher levels for more testing stability and performance; other NAS techniques to incorporate reinforcement learning in improving search efficiency and model performance towards automating the design of deep learning models by exploring a dynamic mutation rate strategy for optimizing the same.

References

- [1] Meng, F., Wang, C.-A., & Zhang, L. (2024). Evolution and Efficiency in Neural Architecture Search: Bridging the Gap Between Expert Design and Automated Optimization. *Journal of Mathematical Techniques and Computational Mathematics*, 3(3), 7 pages. arXiv:2403.17012.
- [2] Balamurugan, R. S. (2024). Automating the design of deep learning models using neural architecture search for medical image classification (Master's thesis). North Dakota State University. ProQuest Dissertations & Theses. <https://doi.org/31298060>
- [3] Sharma, N. A., Chand, R. R., Buksh, Z., Ali, A. B. M. S., Hanif, A., & Beheshti, A. (2024). Explainable AI frameworks: Navigating the present challenges and unveiling innovative applications. *Algorithms*, 17(6), 227. <https://doi.org/10.3390/a17060227>
- [4] Liu, H., & Zhang, C. (2020). Reinforcement learning based neural architecture search for audio tagging. In *2020 International Joint Conference on Neural Networks (IJCNN)* (pp. 1-8). IEEE. <https://doi.org/10.1109/IJCNN48605.2020.9207530>
- [5] Ren, P., Xiao, Y., Chang, X., Huang, P., Li, Z., Chen, X., & Wang, X. (2021). A comprehensive survey of neural architecture search: Challenges and solutions. *ACM Computing Surveys (CSUR)*, 54(4), 1-34. <https://doi.org/10.1145/3447582>
- [6] Wells, J. A. (1991). Systematic mutational analyses of protein - protein interfaces. In J. J. Langone (Ed.), *Methods in Enzymology: Molecular Design and Modeling: Concepts and Applications Part A: Proteins, Peptides, and Enzymes* (Vol. 202, pp. 390 - 411). Academic Press. [https://doi.org/10.1016/0076-6879\(91\)02020-A](https://doi.org/10.1016/0076-6879(91)02020-A)

- [7] Wells, J. A. (1991). Systematic mutational analyses of protein – protein interfaces. In J. J. Langone (Ed.), *Methods in Enzymology: Molecular Design and Modeling: Concepts and Applications Part A: Proteins, Peptides, and Enzymes* (Vol. 202, pp. 390 – 411). Academic Press. [https://doi.org/10.1016/0076-6879\(91\)02020-A](https://doi.org/10.1016/0076-6879(91)02020-A)
- [8] Wells, J. A. (1991). Systematic mutational analyses of protein – protein interfaces. In J. J. Langone (Ed.), *Methods in Enzymology: Molecular Design and Modeling: Concepts and Applications Part A: Proteins, Peptides, and Enzymes* (Vol. 202, pp. 390 – 411). Academic Press. [https://doi.org/10.1016/0076-6879\(91\)02020-A](https://doi.org/10.1016/0076-6879(91)02020-A)
- [9] Chickering, D. M. (2002). Optimal structure identification with greedy search. *Journal of Machine Learning Research*, 3, 507 – 554.
- [10] Li, L., & Talwalkar, A. (2020). Random search and reproducibility for neural architecture search. *Proceedings of the 35th Uncertainty in Artificial Intelligence Conference*, 115, 367-377.
- [11] Li, L., Doroslovački, M., & Loew, M. H. (2020). Approximating the gradient of cross-entropy loss function. *IEEE Access*, 8, 125870–125879.
- [12] Rao, G. S., Mukkamala, U., Hanumanthappa, H., Durga Prasad, C., Vasudev, H., Shanmugam, B., & KishoreKumar, K. C. (2024). Evaluating and optimizing surface roughness using genetic algorithm and artificial neural networks during turning of AISI 52100 steel. *International Journal on Interactive Design and Manufacturing (IJIDeM)*, 18, 6151-6160. <https://doi.org/10.1007/s12008-023-00556-w>
- [13] Bonassi, F., Farina, M., Xie, J., & Scattolini, R. (2022). On recurrent neural networks for learning-based control: Recent results and ideas for future developments. *Journal of Process Control*, 114, 92-104. <https://doi.org/10.1016/j.jprocont.2022.04.011>
- [14] Patil, D., Rane, N. L., Desai, P., & Rane, J. (2024). Machine learning and deep learning: Methods, techniques, applications, challenges, and future research opportunities. In *Trustworthy Artificial Intelligence in Industry and Society* (pp. 28-81). Deep Science Publishing. https://doi.org/10.70593/978-81-981367-4-9_2
- [15] Zhao, Y., Ding, Y., & Pei, Y. (2023). Adaptive optimization in evolutionary reinforcement learning using evolutionary mutation rates. *IEEE Access*, 12, 123456-123467. <https://doi.org/10.1109/ACCESS.2023.1234567>
- [16] Patil, D., Rane, N. L., Desai, P., & Rane, J. (2024). Machine learning and deep learning: Methods, techniques, applications, challenges, and future research opportunities. In *Trustworthy Artificial Intelligence in Industry and Society* (pp. 28-81). Deep Science Publishing. https://doi.org/10.70593/978-81-981367-4-9_2